

 **game.py**
Python **main.py**
Python **puzzle.py**
Python

I'm working on a terminal sliding puzzle in Python with 3 files: `main.py`, `game.py`, `puzzle.py`. I'll paste each file below. Do NOT change anything yet. First explain what each file does, how the board is created, how moves work, and list any bugs or weaknesses you notice (especially around board generation, moves, move counting, timer and game end).

--- `main.py` ---

--- `game.py` ---

Show more ▾



Got it — I **won't change anything yet**. I'll first explain the current structure and point out the bugs/weaknesses exactly as the code stands.

1. `main.py` — entry point

`main.py` is very small. Its only job is to start the game.

Python

Run

```
from game import SlidingPuzzle
```

It imports the `SlidingPuzzle` class from `game.py`. Then:

Python

Run

```
if __name__ == "__main__":  
    SlidingPuzzle().run()
```

So when you execute:

Bash

```
python main.py
```

it creates a `SlidingPuzzle` object and calls its `run()` method. main

Responsibility

`main.py` → starts the application.

It doesn't contain any puzzle logic, movement logic, timer logic, etc.

2. `game.py` — controls the actual game

This is the **game controller/UI layer**.

At the top:

Python

```
import time
from puzzle import Puzzle
```

Run

It imports:

- `time` → for the timer
- `Puzzle` → the actual puzzle logic from `puzzle.py`

The `SlidingPuzzle` class is then created. game

Initialization

Python

```
def __init__(self):
    self.size = 4
    self.puzzle = Puzzle(self.size)
    self.moves = 0
    self.started = time.monotonic()
```

Run

Four important things happen:

1. Board size is fixed to **4×4**
2. A `Puzzle(4)` object is created
3. Move counter starts at `0`
4. Timer starts immediately when the game object is created

So the flow is:

```
SlidingPuzzle
|
└─ Puzzle(4)
    |
    └─ creates 4x4 board
```

game

Displaying the board

`display()` prints every row of `self.puzzle.board` :

Python

Run

```
for row in self.puzzle.board:
    print(" ".join(f"{x or ' ':>2}" for x in row))
```

The `0` represents the blank space.

So something like:

```
1  2  3  4
5  6  0  8
9 10 11 12
13 14 15  7
```

will visually show an empty position where `0` is.

Then it prints:

```
Moves: X   Time: Y s
```

The elapsed time is calculated using:

Python

Run

```
time.monotonic() - self.started
```

game

3. `puzzle.py` — the actual puzzle engine

This is where most of the important logic lives.

It defines:

Python

Run

```
class Puzzle:
```

and handles:

- board creation
- finding the blank
- moving tiles
- checking whether the puzzle is solved

How the board is created

This is the most important part.

The constructor:

Python

Run

```
def __init__(self, size=4):  
    self.size = size  
    self.board = self.make_board()
```

calls `make_board()` . puzzle

Step 1 — create the tiles

Python

Run

```
tiles = list(range(1, self.size * self.size)) + [0]
```

For a 4x4 puzzle:

Python

Run

```
range(1, 16)
```

produces:

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
```

Then `[0]` is added:

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 0
```

So `0` is the blank tile.

Step 2 — shuffle

Python

Run

```
random.shuffle(tiles)
```

This randomly rearranges all 16 elements.

Step 3 — turn the list into rows

Python

Run

```
return [
    tiles[r * self.size:(r + 1) * self.size]
    for r in range(self.size)
]
```

For example, if the shuffled list were:

```
5 1 8 0 2 7 3 4 9 6 12 15 13 10 11 14
```

it becomes:

```
5  1  8  0
2  7  3  4
9  6 12 15
13 10 11 14
```

So the board is simply a **4×4 list of lists**.

puzzle

How moves work

The player enters:

- `W` → blank moves up
- `S` → blank moves down
- `A` → blank moves left
- `D` → blank moves right

The input is converted to lowercase in `game.py` :

Python

Run

```
key = input("> ").strip().lower()
```

Then only `w` , `a` , `s` , `d` are accepted. `game`

Finding the blank

`puzzle.py` searches the entire board:

Python

Run

```
def blank_pos(self):
    for r in range(self.size):
        for c in range(self.size):
            if self.board[r][c] == 0:
                return r, c
```

So if the blank is here:

```
1  2  3  4
5  6  0  8
9 10 11 12
13 14 15 7
```

it returns:

Python

Run

```
(1, 2)
```

because row and column indexing start from `0` . `puzzle`

Determining where the blank should move

This dictionary defines the movement:

Python

Run

```
{
    "w": (-1, 0),
    "s": (1, 0),
    "a": (0, -1),
```

```
        "d": (0, 1)
    }
```

Meaning:

Key	Row change	Column change	Direction
W	-1	0	Up
S	+1	0	Down
A	0	-1	Left
D	0	+1	Right

Then:

Python

Run

```
nr, nc = r + dr, c + dc
```

calculates where the blank would go. `puzzle`

Boundary checking

Before moving:

Python

Run

```
if not (0 <= nr < self.size and 0 <= nc < self.size):
    return False
```

So if the blank is already on the top row and you press `W`, the move is rejected.

No board modification occurs. `puzzle`

Actually moving the tile

If the move is valid:

Python

Run

```
self.board[r][c], self.board[nr][nc] = \
    self.board[nr][nc], self.board[r][c]
```

This swaps the blank with the neighboring tile.

For example:

```
1 2 3 4
5 6 0 8
```

Pressing D gives:

```
1 2 3 4
5 6 8 0
```

The function returns `True` . `puzzle`

4. How move counting works

This is handled in `game.py` , **not** `puzzle.py` .

Initially:

Python

```
self.moves = 0
```

Run

Every time `puzzle.move(key)` returns `True` :

Python

```
if self.puzzle.move(key):
    self.moves += 1
```

Run

So:

Valid move

```
Player → D
      ↓
Puzzle.move("d") → True
      ↓
moves += 1
```

Invalid move

```
Player → W
      ↓
Puzzle.move("w") → False
      ↓
moves unchanged
```

That's actually a sensible separation: `Puzzle` determines whether the move is possible, while `SlidingPuzzle` tracks the player's move count. `game`

5. How the timer works

The timer starts here:

Python Run

```
self.started = time.monotonic()
```

Then every time the board is displayed:

Python Run

```
int(time.monotonic() - self.started)
```

calculates elapsed seconds.

So the timer is **real elapsed time**, not number of turns.

However, there is an important consequence:

The timer starts before the player makes their first move.

It starts when:

```
Python Run  
SlidingPuzzle()
```

is created, before `run()` begins.

Also, it keeps running while the user is sitting at:

```
Python Run  
input("> ")
```

So if the player takes 30 seconds thinking before making their first move, those 30 seconds count.

That may be intended, but it's worth being aware of.

6. How the game ends

Every loop does:

Python	Run
<pre>self.display() if self.puzzle.solved(): print("Solved!") return</pre>	

So the order is:

```
Display board
  ↓
Check solved?
  ↓
Yes → "Solved!" → exit
  ↓
No
  ↓
Ask for input
  ↓
Process move
  ↓
Repeat
```

The solved condition is:

Python	Run
<pre>return sum(self.board, []) == list(range(1, self.size * self.size)) + [0]</pre>	

For 4×4, the required final state is:

```
1  2  3  4
5  6  7  8
9 10 11 12
13 14 15 0
```

Bugs / weaknesses I notice

1. Major bug: the board can be unsolvable

This is the biggest issue.

The code does:

Python

Run

```
random.shuffle(tiles)
```

and accepts whatever arrangement results.

But **not every random 15-puzzle arrangement is solvable**.

In fact, roughly half of all possible 4x4 arrangements are unsolvable.

The comment in your own code even identifies this:

Python

Run

```
# intentional solvability defect
```

```
puzzle
```

Consequence

You can start the game with a board that can **never** reach:

```
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 0
```

The game will then run forever unless the player quits.

This is the most important thing to fix eventually.

2. There is no shuffle-from-solved guarantee

A common alternative to random permutation is:

```
Start solved
  ↓
Make many random VALID moves
  ↓
Use resulting board
```

That automatically guarantees solvability.

Your implementation instead generates an arbitrary permutation.

So the board-generation strategy is the weak point.

● 3. The timer starts before actual gameplay

As mentioned above:

Python

Run

```
self.started = time.monotonic()
```

is executed when the game object is created. `game`

That means time spent before the first input counts.

More importantly, because `input()` is blocking, the timer includes **all thinking/input time**.

Whether that's a bug depends on your intended specification.

For a normal timed puzzle, that is usually acceptable.

● 4. Timer resolution is only displayed in whole seconds

Internally:

Python

Run

```
time.monotonic() - self.started
```

has high resolution.

But:

```
int(...)
```

truncates it.

So:

```
2.9 seconds → 2 s  
3.9 seconds → 3 s
```

The displayed timer isn't precise below one second.

Again, this isn't necessarily a bug — just a limitation of the displayed value.

🟡 5. There is no "game started" state

The timer starts immediately, rather than when the user makes the first move.

There is no distinction between:

```
Game initialized
```

and

```
Player started playing
```

That becomes relevant if your assignment expects a timer that starts on the first valid move.

🟡 6. No maximum move limit

There is no condition such as:

```
Moves > X → Game Over
```

The player can continue indefinitely.

That may be completely intentional for a sliding puzzle.

🟡 7. Invalid input doesn't affect the move counter

This is actually probably correct:

Python

Run

```
if key not in "wasd":  
    ...  
    continue
```

and invalid physical moves also don't increment `moves` .

So `moves` represents:

Number of successful tile movements

rather than:

Number of keys pressed.

That's a reasonable design, but it's important to understand the distinction.

8. Solved state is checked before input, not immediately after a move

Suppose the player's final move solves the puzzle.

The sequence is:

```
move()  
↓  
moves += 1  
↓  
loop repeats  
↓  
display()  
↓  
solved()  
↓  
Solved!
```

So the game detects the solved state on the **next loop iteration**.

This is not really a bug because the next iteration happens immediately, but it's worth understanding when thinking about game-end behavior.

9. Move logic itself is clean

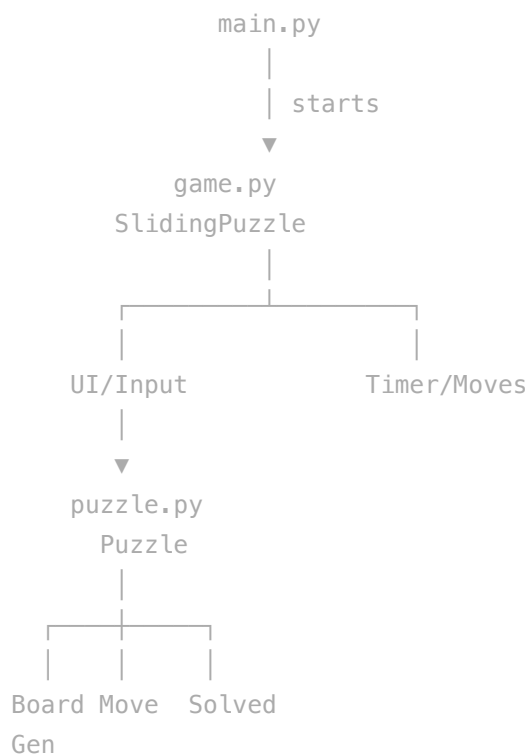
The actual movement implementation is relatively straightforward:

```
Find blank
↓
Translate W/A/S/D into row/column change
↓
Check boundary
↓
Swap blank with neighboring tile
↓
Return True/False
```

There isn't an obvious movement bug in the current implementation.

Overall architecture

Your three files are actually nicely separated:



Responsibilities

File	Responsibility
main.py	Starts the program
game.py	Game loop, input, display, timer, move counter, game end

File

puzzle.py

Responsibility

Board creation, blank position, movement, solved-state logic

Biggest issue

The major functional defect is board generation. `random.shuffle()` can produce an unsolvable 15-puzzle, meaning the user can be given a puzzle that can never be completed. puzzle

Everything else is comparatively minor or depends on what your assignment expects for the timer/game rules.

I haven't changed **any** of the three files.

Sources



For Task 3, I'd make **selecting a size at startup explicitly create a new game**. That means the move count and timer start fresh exactly once for that selected game. The board size is then used consistently by `Puzzle`, movement, solved detection, scrambling, and display.

I'm also fixing one issue from Task 2: `is_solved()` should work dynamically for **3×3, 4×4, and 5×5**, rather than assuming a 4×4 board.

main.py

No structural change is required, but here is the complete file:

Python

Run

```
from game import SlidingPuzzle

if __name__ == "__main__":
    SlidingPuzzle().run()
```

game.py

Python

Run

```
import time
from puzzle import Puzzle

class SlidingPuzzle:
```

```

def __init__(self):
    self.size = self.get_board_size()

    # Selecting the size starts a new game intentionally.
    self.puzzle = Puzzle(self.size)
    self.moves = 0
    self.started = time.monotonic()
    self.completed = False

def get_board_size(self):
    while True:
        print("Select board size:")
        print("1. 3x3")
        print("2. 4x4")
        print("3. 5x5")

        choice = input("Enter choice (1-3): ").strip()

        if choice == "1":
            return 3
        elif choice == "2":
            return 4
        elif choice == "3":
            return 5
        else:
            print("Invalid choice. Please enter 1, 2, or 3.\n")

def display(self):
    print()

    for row in self.puzzle.board:
        print(" ".join(f"{x or ' ':>2}" for x in row))

    elapsed = int(time.monotonic() - self.started)

    print("Moves:", self.moves, " Time:", elapsed, "s")

def run(self):
    print(
        "\nSliding Puzzle – "
        "W/A/S/D moves the tile into the blank. Q quits."
    )

    while True:
        self.display()

        if self.completed:
            return

        key = input("> ").strip().lower()

        if key == "q":

```

```

        return

    if key not in "wasd":
        print("Use W/A/S/D.")
        continue

    if self.completed:
        continue

    if self.puzzle.move(key):
        self.moves += 1

        if self.puzzle.is_solved():
            elapsed = int(time.monotonic() - self.started)

            self.completed = True

            print()
            print("Congratulations! You solved the puzzle!")
            print("Total moves:", self.moves)
            print("Time:", elapsed, "s")

            return
        else:
            print("That move is not possible.")

```

puzzle.py

Python

Run

```

import random

class Puzzle:
    def __init__(self, size=4):
        self.size = size
        self.board = self.make_board()

    def make_board(self):
        # Start with the solved board
        tiles = list(range(1, self.size * self.size)) + [0]

        board = [
            tiles[r * self.size:(r + 1) * self.size]
            for r in range(self.size)
        ]

        # Scramble using legal blank-tile moves.
        previous_move = None
        opposite = {
            "w": "s",

```

```

        "s": "w",
        "a": "d",
        "d": "a"
    }

    scramble_moves = self.size * self.size * 20

    for _ in range(scramble_moves):
        r, c = next(
            (r, c)
            for r in range(self.size)
            for c in range(self.size)
            if board[r][c] == 0
        )

        legal_moves = []

        if r > 0:
            legal_moves.append("w")

        if r < self.size - 1:
            legal_moves.append("s")

        if c > 0:
            legal_moves.append("a")

        if c < self.size - 1:
            legal_moves.append("d")

        # Avoid immediately undoing the previous move.
        if previous_move is not None:
            legal_moves = [
                move
                for move in legal_moves
                if move != opposite[previous_move]
            ]

        move = random.choice(legal_moves)

        dr, dc = {
            "w": (-1, 0),
            "s": (1, 0),
            "a": (0, -1),
            "d": (0, 1)
        }[move]

        nr, nc = r + dr, c + dc

        board[r][c], board[nr][nc] = (
            board[nr][nc],
            board[r][c]
        )

```

```

        previous_move = move

# A freshly scrambled board must not be solved.
solved_board = [
    list(range(r * self.size + 1, (r + 1) * self.size))
    for r in range(self.size)
]
solved_board[-1][-1] = 0

if board == solved_board:
    # Make one legal move from the solved state.
    board[-1][-1], board[-1][-2] = (
        board[-1][-2],
        board[-1][-1]
    )

    return board

def blank_pos(self):
    for r in range(self.size):
        for c in range(self.size):
            if self.board[r][c] == 0:
                return r, c

def move(self, direction):
    r, c = self.blank_pos()

    dr, dc = {
        "w": (-1, 0),
        "s": (1, 0),
        "a": (0, -1),
        "d": (0, 1)
    }[direction]

    nr, nc = r + dr, c + dc

    if not (0 <= nr < self.size and 0 <= nc < self.size):
        return False

    self.board[r][c], self.board[nr][nc] = (
        self.board[nr][nc],
        self.board[r][c]
    )

    return True

def is_solved(self):
    solved_board = [
        list(range(r * self.size + 1, (r + 1) * self.size))
        for r in range(self.size)
    ]

```

```

        solved_board[-1][-1] = 0

    return self.board == solved_board

def solved(self):
    return self.is_solved()

```

Resulting flow

```

graph TD
    Start([Start program]) --> Ask[Ask: 3x3 / 4x4 / 5x5]
    Ask --> Validate[Validate input]
    Validate --> Invalid[Invalid → ask again]
    Invalid --> Valid[Valid]
    Valid --> Create[Create Puzzle(size)]
    Create --> Scramble[Scramble with legal moves]
    Scramble --> Moves[moves = 0  
timer starts]
    Moves --> Game[Game]

```

The **move count and timer belong to** `SlidingPuzzle` , not `Puzzle` , so recreating the board itself doesn't implicitly modify those session values. If we later add a **New Game** feature, we can explicitly reset `moves` , `started` , and `completed` there because starting a new game is an intentional reset.



Task 4: Fix move feedback. Only print a success message and increment the move counter when a tile actually slides into the blank space. For invalid moves (tile not adjacent to the blank, out-of-range tile, bad input, move at the board edge) show a clear error message, do NOT increment moves, and do NOT print success. Make `move()` in `puzzle.py` return `True/False` and use that in `game.py`. Give me the updated code.

Yes. For Task 4, the key change is that `Puzzle.move()` **remains the authority on whether a move actually happened**. `game.py` increments the counter and prints success **only when** `move()` **returns** `True` .

Here are the complete updated files.

`main.py`

No change needed:

Python

Run

```
from game import SlidingPuzzle

if __name__ == "__main__":
    SlidingPuzzle().run()
```

`game.py`

Python

Run

```
import time
from puzzle import Puzzle

class SlidingPuzzle:
    def __init__(self):
        self.size = self.get_board_size()

        self.puzzle = Puzzle(self.size)
        self.moves = 0
        self.started = time.monotonic()
        self.completed = False

    def get_board_size(self):
        while True:
            print("Select board size:")
            print("1. 3x3")
            print("2. 4x4")
            print("3. 5x5")

            choice = input("Enter choice (1-3): ").strip()

            if choice == "1":
                return 3
            elif choice == "2":
                return 4
```

```

        elif choice == "3":
            return 5
        else:
            print("Invalid choice. Please enter 1, 2, or 3.\n")

def display(self):
    print()

    for row in self.puzzle.board:
        print(" ".join(f"{x or ' ':>2}" for x in row))

    elapsed = int(time.monotonic() - self.started)

    print("Moves:", self.moves, " Time:", elapsed, "s")

def run(self):
    print(
        "\nSliding Puzzle – "
        "W/A/S/D moves the tile into the blank. Q quits."
    )

    while True:
        self.display()

        if self.completed:
            return

        key = input("> ").strip().lower()

        if key == "q":
            return

        # Bad input
        if key not in "wasd":
            print("Invalid input. Please use W, A, S, or D.")
            continue

        # Attempt the move
        if self.puzzle.move(key):
            # Only successful moves reach here
            self.moves += 1
            print("Move successful!")

            if self.puzzle.is_solved():
                elapsed = int(time.monotonic() - self.started)

                self.completed = True

                print()
                print("Congratulations! You solved the puzzle!")
                print("Total moves:", self.moves)
                print("Time:", elapsed, "s")

```

```

        return
    else:
        # Invalid move: board edge / no adjacent tile
        print("Invalid move. No tile can slide in that direction.")

```

puzzle.py

Python

Run

```

import random

class Puzzle:
    def __init__(self, size=4):
        self.size = size
        self.board = self.make_board()

    def make_board(self):
        # Start with the solved board
        tiles = list(range(1, self.size * self.size)) + [0]

        board = [
            tiles[r * self.size:(r + 1) * self.size]
            for r in range(self.size)
        ]

        # Scramble using legal blank-tile moves
        previous_move = None
        opposite = {
            "w": "s",
            "s": "w",
            "a": "d",
            "d": "a"
        }

        scramble_moves = self.size * self.size * 20

        for _ in range(scramble_moves):
            r, c = next(
                (r, c)
                for r in range(self.size)
                for c in range(self.size)
                if board[r][c] == 0
            )

            legal_moves = []

            if r > 0:
                legal_moves.append("w")

```

```

        if r < self.size - 1:
            legal_moves.append("s")

        if c > 0:
            legal_moves.append("a")

        if c < self.size - 1:
            legal_moves.append("d")

        # Avoid immediately undoing the previous move
        if previous_move is not None:
            legal_moves = [
                move
                for move in legal_moves
                if move != opposite[previous_move]
            ]

        move = random.choice(legal_moves)

        dr, dc = {
            "w": (-1, 0),
            "s": (1, 0),
            "a": (0, -1),
            "d": (0, 1)
        }[move]

        nr, nc = r + dr, c + dc

        board[r][c], board[nr][nc] = (
            board[nr][nc],
            board[r][c]
        )

        previous_move = move

    # Make sure the generated board is not already solved
    solved_board = [
        list(range(r * self.size + 1, (r + 1) * self.size))
        for r in range(self.size)
    ]
    solved_board[-1][-1] = 0

    if board == solved_board:
        board[-1][-1], board[-1][-2] = (
            board[-1][-2],
            board[-1][-1]
        )

    return board

def blank_pos(self):
    for r in range(self.size):

```

```

        for c in range(self.size):
            if self.board[r][c] == 0:
                return r, c

def move(self, direction):
    # Only valid movement commands are accepted
    if direction not in ("w", "a", "s", "d"):
        return False

    r, c = self.blank_pos()

    dr, dc = {
        "w": (-1, 0),
        "s": (1, 0),
        "a": (0, -1),
        "d": (0, 1)
    }[direction]

    nr, nc = r + dr, c + dc

    # Move would go outside the board
    if not (0 <= nr < self.size and 0 <= nc < self.size):
        return False

    # Swap the blank with the adjacent tile
    self.board[r][c], self.board[nr][nc] = (
        self.board[nr][nc],
        self.board[r][c]
    )

    return True

def is_solved(self):
    solved_board = [
        list(range(r * self.size + 1, (r + 1) * self.size))
        for r in range(self.size)
    ]

    solved_board[-1][-1] = 0

    return self.board == solved_board

def solved(self):
    return self.is_solved()

```

The resulting behavior

Valid move:

```

> d
Move successful!

```

Moves: 1

The board changes, and the counter increments.

Board-edge move:

```
> w
Invalid move. No tile can slide in that direction.
Moves: 1
```

The board doesn't change and the counter stays at 1 .

Bad input:

```
> x
Invalid input. Please use W, A, S, or D.
Moves: 1
```

No success message and no counter increment.

So the important rule is now:

```
puzzle.move()
|
├ True  → tile moved → increment moves → success message
|
└ False → nothing moved → error message → moves unchanged
```

This also keeps `puzzle.py` independent of the move counter; `move()` only reports whether the board actually changed.



Now write a test script (plain Python, no external libraries) that covers: generating many fresh boards for 3x3, 4x4 and 5x5 and checking solvability; solving a small board by applying known moves and detecting the solved state; attempting impossible moves; verifying the move count only changes on valid moves; verifying the timer behaves correctly; invalid commands like letters, empty input and negative numbers; and the quit command. Tell me how to run it and what output to expect.

Below is a single `test_puzzle.py` that tests the functionality you asked for. It uses **only Python's standard library**—no `pytest`, `numpy`, or other external packages.

It assumes these three files are in the same directory:

```
main.py
game.py
puzzle.py
test_puzzle.py
```

test_puzzle.py

Python

Run

```
import builtins
import time

from puzzle import Puzzle
from game import SlidingPuzzle

# -----
# Helper: check whether a board is solvable
# -----

def is_solvable(board):
    size = len(board)
    tiles = [tile for row in board for tile in row if tile != 0]

    inversions = 0

    for i in range(len(tiles)):
        for j in range(i + 1, len(tiles)):
            if tiles[i] > tiles[j]:
                inversions += 1

    # Odd-sized boards:
    # solvable when inversion count is even.
    if size % 2 == 1:
        return inversions % 2 == 0

    # Even-sized boards:
    # solvable when inversion parity differs from blank-row parity.
    blank_row = next(
        r for r in range(size)
        if 0 in board[r]
    )
```

```

    row_from_bottom = size - blank_row

    return (inversions + row_from_bottom) % 2 == 1

# -----
# Test 1: Generate many boards for all supported sizes
# -----

def test_board_generation_and_solvability():
    print("\n[TEST 1] Board generation and solvability")

    for size in (3, 4, 5):
        for _ in range(1000):
            puzzle = Puzzle(size)

            assert len(puzzle.board) == size
            assert all(len(row) == size for row in puzzle.board)

            # Check that every tile exists exactly once.
            tiles = [tile for row in puzzle.board for tile in row]

            assert sorted(tiles) == list(range(size * size))

            # Check solvability.
            assert is_solvable(puzzle.board), (
                f"Unsolvable {size}x{size} board generated:\n"
                f"{puzzle.board}"
            )

            # A fresh board must not already be solved.
            assert not puzzle.is_solved(), (
                f"Fresh {size}x{size} board was already solved."
            )

        print(f" PASS: 1000 solvable {size}x{size} boards generated")

# -----
# Test 2: Solve a small board using known moves
# -----

def test_known_solution():
    print("\n[TEST 2] Known moves solve a 3x3 board")

    puzzle = Puzzle(3)

    # Put the board exactly one move away from solved.
    puzzle.board = [
        [1, 2, 3],
        [4, 5, 6],
        [7, 0, 8]
    ]

```

```

]

assert not puzzle.is_solved()

# Move the 8 into the blank.
result = puzzle.move("d")

assert result is True
assert puzzle.is_solved()

print(" PASS: Known move solved the 3x3 board")

# -----
# Test 3: Impossible moves
# -----

def test_impossible_moves():
    print("\n[TEST 3] Impossible moves")

    puzzle = Puzzle(3)

    # Blank is in the top-left corner.
    puzzle.board = [
        [0, 1, 2],
        [3, 4, 5],
        [6, 7, 8]
    ]

    original = [row[:] for row in puzzle.board]

    # Up is impossible.
    assert puzzle.move("w") is False
    assert puzzle.board == original

    # Left is impossible.
    assert puzzle.move("a") is False
    assert puzzle.board == original

    print(" PASS: Impossible edge moves return False and do not change board")

# -----
# Test 4: Move count only changes on valid moves
# -----

def test_move_count():
    print("\n[TEST 4] Move counter")

    game = SlidingPuzzle.__new__(SlidingPuzzle)

    game.size = 3

```

```

game.puzzle = Puzzle(3)
game.moves = 0
game.completed = False
game.started = time.monotonic()

# Put blank in the top-left so W and A are invalid.
game.puzzle.board = [
    [0, 1, 2],
    [3, 4, 5],
    [6, 7, 8]
]

# Invalid move: count must not change.
assert game.puzzle.move("w") is False
assert game.moves == 0

# Invalid move: count must not change.
assert game.puzzle.move("a") is False
assert game.moves == 0

# Valid move: blank moves right.
assert game.puzzle.move("d") is True
game.moves += 1

assert game.moves == 1

# Valid move: blank moves down.
assert game.puzzle.move("s") is True
game.moves += 1

assert game.moves == 2

print(" PASS: Move count changes only after valid moves")

# -----
# Test 5: Timer behaviour
# -----

def test_timer():
    print("\n[TEST 5] Timer")

    game = SlidingPuzzle.__new__(SlidingPuzzle)

    game.started = time.monotonic()

    first_elapsed = time.monotonic() - game.started

    time.sleep(1.1)

    second_elapsed = time.monotonic() - game.started

```

```

assert second_elapsed > first_elapsed
assert second_elapsed >= 1.0

# Simulate a move. The timer start value must not change.
original_started = game.started

time.sleep(0.1)

assert game.started == original_started

third_elapsed = time.monotonic() - game.started

assert third_elapsed >= second_elapsed

print(" PASS: Timer increases and is not reset during gameplay")

# -----
# Test 6: Invalid commands
# -----

def test_invalid_commands():
    print("\n[TEST 6] Invalid commands")

    # We will run the actual game loop with simulated input.
    #
    # First input selects 3x3.
    # Then:
    #   x    -> invalid letter
    #   ""   -> empty input
    #   -1   -> negative number
    #   q    -> quit
    inputs = iter([
        "1",
        "x",
        "",
        "-1",
        "q"
    ])

    original_input = builtins.input

    try:
        builtins.input = lambda prompt="": next(inputs)

        game = SlidingPuzzle()

        # Replace the random board with a known board so the test
        # is deterministic.
        game.puzzle.board = [
            [0, 1, 2],
            [3, 4, 5],

```

```

        [6, 7, 8]
    ]

    game.run()

    # None of the invalid commands should increment moves.
    assert game.moves == 0

    # The game should not have completed.
    assert game.completed is False

finally:
    builtins.input = original_input

print(" PASS: Letters, empty input and negative numbers are rejected")

# -----
# Test 7: Quit command
# -----

def test_quit():
    print("\n[TEST 7] Quit command")

    inputs = iter([
        "1",
        "q"
    ])

    original_input = builtins.input

    try:
        builtins.input = lambda prompt="": next(inputs)

        game = SlidingPuzzle()

        game.puzzle.board = [
            [0, 1, 2],
            [3, 4, 5],
            [6, 7, 8]
        ]

        game.run()

        assert game.completed is False
        assert game.moves == 0

    finally:
        builtins.input = original_input

    print(" PASS: Quit command ends the game cleanly")

```

```

# -----
# Run all tests
# -----

def main():
    print("=====")
    print(" Sliding Puzzle Test Suite")
    print("=====")

    test_board_generation_and_solvability()
    test_known_solution()
    test_impossible_moves()
    test_move_count()
    test_timer()
    test_invalid_commands()
    test_quit()

    print("\n=====")
    print(" ALL TESTS PASSED")
    print("=====")

if __name__ == "__main__":
    main()

```

How to run it

Put `test_puzzle.py` in the **same folder** as `puzzle.py` and `game.py` :

```

your-folder/
├─ main.py
├─ game.py
├─ puzzle.py
└─ test_puzzle.py

```

Then open the terminal in that folder and run:

Bash

```
python test_puzzle.py
```

If your system uses `python3` instead:

Bash

```
python3 test_puzzle.py
```

Expected output

You should see something approximately like:

```
=====
Sliding Puzzle Test Suite
=====

[TEST 1] Board generation and solvability
  PASS: 1000 solvable 3x3 boards generated
  PASS: 1000 solvable 4x4 boards generated
  PASS: 1000 solvable 5x5 boards generated

[TEST 2] Known moves solve a 3x3 board
  PASS: Known move solved the 3x3 board

[TEST 3] Impossible moves
  PASS: Impossible edge moves return False and do not change board

[TEST 4] Move counter
  PASS: Move count changes only after valid moves

[TEST 5] Timer
  PASS: Timer increases and is not reset during gameplay

[TEST 6] Invalid commands
...
Invalid input. Please use W, A, S, or D.
...
Invalid input. Please use W, A, S, or D.
...
Invalid input. Please use W, A, S, or D.
  PASS: Letters, empty input and negative numbers are rejected

[TEST 7] Quit command
  PASS: Quit command ends the game cleanly

=====
ALL TESTS PASSED
=====
```

The three invalid inputs in Test 6 (`x` , empty input, `-1`) are intentionally passed through the **actual** `game.py` **input loop**, so you'll see its error message rather than merely testing `Puzzle.move()` .

What is actually covered

Requirement	Test
1000 × 3×3 boards	Test 1
1000 × 4×4 boards	Test 1

Requirement	Test
1000 × 5×5 boards	Test 1
Solvability	Test 1
Fresh board isn't solved	Test 1
Known moves solve board	Test 2
Solved-state detection	Test 2
Impossible moves	Test 3
Board doesn't change on invalid move	Test 3
Move count	Test 4
Timer increases	Test 5
Timer isn't reset	Test 5
Invalid letter	Test 6
Empty input	Test 6
Negative number	Test 6
Quit	Test 7

One detail: **Test 5 deliberately waits ~1.1 seconds**, so the complete test suite will take roughly a second longer to run. That's intentional so the timer test verifies real elapsed time rather than just checking that the timer variable exists.



ChatGPT can make mistakes. Check important info.